Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Национальный исследовательский университет «МЭИ»

Институт тепловой и атомной энергетики

Кафедра общей физики и ядерного синтеза

ОТЧЕТ
О ПРАКТИЧЕСКОЙ РАБОТЕ № 1

по дисциплине «Имитационное моделирование плазменных процессов»

РЕШЕНИЕ ЗАДАЧИ ДВУХ ТЕЛ

Вариант 4

Студент Голов Я. А.

Преподаватель Лубенченко А. В.

Москва 2026

РЕФЕРАТ

Отчет 43 с., 8 рис., 5 табл., 7 источн., 1 прил.

ЗАДАЧА ДВУХ ТЕЛ, УПРУГОЕ РАССЕЯНИЕ, ЭКРАНИРОВАННЫЙ КУЛОНОВСКИЙ ПОТЕНЦИАЛ, ПОТЕНЦИАЛ KR-C, РАДИУС ЭКРАНИРОВАНИЯ ФИРСОВА, ПРИЦЕЛЬНЫЙ ПАРАМЕТР, КИНЕМАТИЧЕСКИЙ ФАКТОР, КВАДРАТУРА ГАУССА – МЕЛЕРА, ДИФФЕРЕНЦИАЛЬНОЕ СЕЧЕНИЕ РАССЕЯНИЯ

Объектом исследования является упругое парное столкновение протона с покоящимся атомом меди при энергии налетающей частицы 0,1 эВ.

Цель работы — определение зависимости угла рассеяния частицы от прицельного параметра при столкновении двух частиц и зависимости дифференциального сечения упругого рассеяния от углов рассеяния.

Методы исследования: классическая теория рассеяния двух тел с экранированным кулоновским потенциалом Kr-C, численное решение нелинейного уравнения методом деления отрезка пополам, численное интегрирование методом Гаусса – Мелера, численное дифференцирование. Программа написана на языке Python с применением объектно-ориентированного подхода.

В результате работы рассчитаны зависимости потенциальной энергии взаимодействия от расстояния, кинематического фактора и энергии рассеянной частицы от угла рассеяния, расстояния наименьшего сближения, угла рассеяния в системе центра масс и углов рассеяния в лабораторной системе отсчета от прицельного параметра, а также дифференциальные сечения упругого рассеяния. Правильность программы подтверждена сравнением с точным решением задачи рассеяния в кулоновском поле.

Результаты могут быть использованы при моделировании взаимодействия ионов с веществом методами молекулярной динамики и Монте-Карло.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1 Физическая модель 6
2 Математическая модель 7
2.1 Потенциал взаимодействия 7
2.2 Кинематика упругого столкновения 8
2.3 Угол рассеяния и сечение 8
3 Дискретная модель и алгоритм решения 10
3.1 Расстояние наименьшего сближения 10
3.2 Сетка прицельных параметров 10
3.3 Численное интегрирование 10
3.4 Численное дифференцирование 11
3.5 Алгоритм решения 11
4 Программная реализация 12
5 Тестовые расчеты 14
6 Результаты моделирования 16
6.1 Потенциальная энергия и кинематический фактор 16
6.2 Сетка прицельных параметров 17
6.3 Углы рассеяния 18
6.4 Дифференциальные сечения рассеяния 19
7 Анализ результатов 21
ЗАКЛЮЧЕНИЕ 22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 23
ПРИЛОЖЕНИЕ А Текст программы 24

ВВЕДЕНИЕ

Расчет парных столкновений лежит в основе имитационного моделирования взаимодействия ускоренных частиц с веществом: и в методе молекулярной динамики, и в методе Монте-Карло результат отдельного столкновения определяется углом рассеяния и потерей энергии, которые зависят от потенциала взаимодействия и прицельного параметра [1.].

Цель работы — определение зависимости угла рассеяния частицы от прицельного параметра при столкновении двух частиц и зависимости дифференциального сечения упругого рассеяния от углов рассеяния.

Задание: рассчитать зависимости угла рассеяния в системе центра масс от прицельного параметра, зависимости углов рассеяния в лабораторной системе отсчета от прицельного параметра, зависимости дифференциального сечения упругого рассеяния от углов рассеяния в лабораторной системе отсчета. При расчете взаимодействия ионов использовать экранированные потенциалы рассеяния.

Исходные данные варианта 4 (сочетание вариантов А = 3 и Б = 1) приведены в таблице 1.

Таблица 1 – Исходные данные варианта 4

Параметр	Значение
Потенциал взаимодействия	Kr-C
Радиус экранирования	Фирсова
Энергия налетающей частицы E_0, эВ	0,1
Налетающая частица	протон p ($Z_1 = 1$, $m_1 = 1{,}00728$ а. е. м.)
Покоящаяся частица	атом меди Cu ($Z_2 = 29$, $m_2 = 63{,}546$ а. е. м.)

Для достижения цели необходимо решить следующие задачи:

– построить зависимость потенциальной энергии взаимодействия $U(r)$ от расстояния между частицами;

– рассчитать зависимости кинематического фактора k и энергии налетающей частицы после рассеяния E_1 от угла рассеяния θ_1;

– рассчитать зависимость расстояния наименьшего сближения R от прицельного параметра ρ и построить сетку прицельных параметров;

– рассчитать зависимость угла рассеяния в системе центра масс χ и углов рассеяния в лабораторной системе θ_1, θ_2 от прицельного параметра;

– рассчитать дифференциальные сечения упругого рассеяния $\omega(\theta_1)$ и $\omega(\theta_2)$;

– разработать программу на языке Python с применением объектно-ориентированного подхода и проверить ее на тестовой задаче.

1 Физическая модель

Рассматривается упругое столкновение двух частиц: налетающей частицы массой m_1 с кинетической энергией E_0 и покоящейся частицы-мишени массой m_2. Принимаются следующие допущения [1., 2., 3.]:

– внешние силы на систему не действуют, поэтому выполняются законы сохранения энергии, импульса и момента импульса;

– потенциал взаимодействия зависит только от расстояния между частицами, поэтому движение происходит в одной плоскости;

– потери энергии на возбуждение и ионизацию электронных оболочек (неупругие потери) не учитываются;

– движение частиц описывается классической механикой.

Применимость классического описания оценивается по длине волны де Бройля налетающей частицы [1.]. Для протона с энергией 0,1 эВ $\lambda_{\text{Б}} = 2{,}87 \cdot 10^{-2}/\sqrt{ME_0} \approx 0{,}09$ нм, что в три раза меньше расстояния наименьшего сближения при лобовом столкновении ($\approx 0{,}28$ нм, см. раздел 6). Следовательно, вариант задания находится вблизи нижней границы применимости классического подхода, но не выходит за нее.

Расстояние наименьшего сближения частиц при рассматриваемых энергиях велико по сравнению с размерами ядер, поэтому важную роль играет экранирование заряда ядер электронными оболочками. Взаимодействие описывается экранированным кулоновским потенциалом — кулоновским потенциалом ядер, умноженным на функцию экранирования, которая учитывает действие электронных оболочек.

Задача двух тел сводится к задаче о движении одной частицы с приведенной массой $\mu = m_1 m_2/(m_1 + m_2)$ в центральном поле $U(r)$. Угол рассеяния в системе центра масс (ц-системе) определяется потенциалом, относительной энергией и прицельным параметром ρ; углы рассеяния в лабораторной системе (л-системе), связанной с покоящейся до удара мишенью, находятся из кинематических соотношений. Для пучка частиц с равномерным распределением по прицельным параметрам вероятность рассеяния в заданный телесный угол характеризуется дифференциальным сечением рассеяния.

2 Математическая модель

2.1 Потенциал взаимодействия

Потенциальная энергия взаимодействия частиц описывается экранированным кулоновским потенциалом [1.]

$$U(r) = \frac{Z_1 Z_2 e^2}{r} F\left(\frac{r}{a}\right), \tag{1}$$

где Z_1, Z_2 — порядковые номера налетающей частицы и мишени;

$e^2 = 1{,}44$ эВ·нм — квадрат элементарного заряда;

r — расстояние между частицами, нм;

F — функция экранирования;

a — длина (радиус) экранирования, нм.

Функция экранирования представляется суммой экспонент

$$F(x) = \sum_{i=1}^{n} c_i \exp(-d_i x), \qquad \sum_{i=1}^{n} c_i = 1, \tag{2}$$

где c_i, d_i — константы потенциала.

Для потенциала Kr-C [4.] $n = 3$, константы приведены в таблице 2.

Таблица 2 – Константы функции экранирования потенциала Kr-C

i	c_i	d_i
1	0,190945	0,278544
2	0,473674	0,637174
3	0,335381	1,919249

Радиус экранирования Фирсова [1.]

$$a_{\text{Ф}} = \left(\frac{9\pi^2}{128}\right)^{1/3} a_{\text{Б}} \left(Z_1^{1/2} + Z_2^{1/2}\right)^{-2/3} \approx 0{,}8853\, a_{\text{Б}} \left(Z_1^{1/2} + Z_2^{1/2}\right)^{-2/3}, \tag{3}$$

где $a_{\text{Б}} = 0{,}0529$ нм — первый боровский радиус.

Для пары p – Cu $a_{\text{Ф}} = 0{,}013608$ нм.

2.2 Кинематика упругого столкновения

Из законов сохранения энергии и импульса следует выражение для кинематического фактора — отношения энергии налетающей частицы после рассеяния E_1 к начальной энергии [1., 2.]:

$$k(\theta_1) = \frac{E_1}{E_0} = \left(\frac{m_1}{m_1 + m_2}\right)^2 \left(\cos\theta_1 + \sqrt{\left(\frac{m_2}{m_1}\right)^2 - \sin^2\theta_1}\right)^2, \qquad (4)$$

где θ_1 — угол рассеяния налетающей частицы в л-системе.

Так как $m_1 < m_2$, угол θ_1 может принимать любые значения от 0 до π, а энергия после рассеяния $E_1 = kE_0$ изменяется от E_0 до $E_0\left((m_2 - m_1)/(m_1 + m_2)\right)^2$.

Энергия относительного движения (энергия в ц-системе)

$$E_r = E_0 \frac{m_2}{m_1 + m_2}. \qquad (5)$$

Для варианта 4 $E_r = 0{,}098440$ эВ.

2.3 Угол рассеяния и сечение

Угол рассеяния в ц-системе [1., 2.]

$$\chi(\rho) = \pi - 2\rho \int_R^\infty \frac{dr}{r^2\sqrt{g(r)}}, \qquad g(r) = 1 - \frac{U(r)}{E_r} - \frac{\rho^2}{r^2}, \qquad (6)$$

где ρ — прицельный параметр, нм;

R — расстояние наименьшего сближения, нм, определяемое как наибольший корень уравнения

$$g(R) = 1 - \frac{U(R)}{E_r} - \frac{\rho^2}{R^2} = 0. \qquad (7)$$

Углы рассеяния налетающей частицы θ_1 и частицы отдачи θ_2 в л-системе связаны с углом χ соотношениями

$$\operatorname{tg}\theta_1 = \frac{m_2 \sin\chi}{m_1 + m_2\cos\chi}, \qquad \theta_2 = \frac{\pi - \chi}{2}. \qquad (8)$$

Дифференциальное сечение упругого рассеяния

$$\omega(\theta) = \frac{\rho(\theta)}{\sin\theta}\left|\frac{d\rho(\theta)}{d\theta}\right|, \tag{9}$$

где $\rho(\theta)$ — функция, обратная зависимости угла рассеяния от прицельного параметра;

θ — угол рассеяния θ_1 или θ_2.

3 Дискретная модель и алгоритм решения

3.1 Расстояние наименьшего сближения

Для отталкивающего потенциала (1) оба слагаемых $-U(r)/E_r$ и $-\rho^2/r^2$ монотонно возрастают с ростом r, поэтому функция $g(r)$ монотонно возрастает от $-\infty$ при $r \to 0$ до 1 при $r \to \infty$, и уравнение (7) имеет единственный корень. Корень находится методом деления отрезка пополам:

1) начальная правая граница $r_+ = \max(\rho, 10^{-3}\text{ нм})$ удваивается, пока $g(r_+) < 0$;

2) начальная левая граница $r_- = r_+$ уменьшается вдвое, пока $g(r_-) \geqslant 0$;

3) отрезок $[r_-, r_+]$ делится пополам до достижения относительной точности 10^{-14}; в качестве R принимается правая граница, на которой $g(R) \geqslant 0$, что гарантирует неотрицательность подкоренного выражения в (6).

3.2 Сетка прицельных параметров

При больших прицельных параметрах потенциал не влияет на движение частиц и $R \to \rho$. Максимальный прицельный параметр $\rho_{\max}$ — наименьшее значение, при котором

$$\left| 1 - \frac{R(\rho_{\max})}{\rho_{\max}} \right| < 10^{-3}. \tag{10}$$

Величина $|1 - R/\rho|$ монотонно убывает с ростом ρ, поэтому $\rho_{\max}$ находится так же, как корень (7): начиная с $\rho = R(0)$, прицельный параметр удваивается до выполнения условия (10), после чего граница уточняется делением отрезка пополам. Сетка прицельных параметров

$$\rho_j = j\,\Delta\rho, \qquad \Delta\rho = \frac{\rho_{\max}}{200}, \qquad j = 1, 2, \ldots, 200. \tag{11}$$

3.3 Численное интегрирование

Подынтегральная функция в (6) имеет интегрируемую особенность в точке $r = R$, где $g(R) = 0$. Замена переменной $r = R/\xi^2$ переводит интеграл на отрезок $\xi \in [0, 1]$, а последующая замена $\xi = \cos\psi$ устраняет особенность.

Применение к полученному интегралу квадратурной формулы средних прямоугольников по ψ дает формулу Гаусса – Мелера [1.]

$$\chi = \pi - 2\,\frac{\rho}{R}\,\frac{\pi}{N}\sum_{n=1}^{N}\frac{\xi_n \xi_{N-n+1}}{\sqrt{g\,(R/\xi_n^2)}}, \qquad \xi_n = \cos\frac{(2n-1)\pi}{4N}, \tag{12}$$

где N — число узлов квадратуры; в расчетах $N = 100$.

3.4 Численное дифференцирование

Зависимости $\rho(\theta_1)$ и $\rho(\theta_2)$ получаются обращением табличных зависимостей $\theta_1(\rho_j)$, $\theta_2(\rho_j)$: значения упорядочиваются по возрастанию угла. Сетка по углу неравномерная, поэтому производная в (9) вычисляется по центральной разностной формуле второго порядка для неравномерной сетки

$$\left.\frac{d\rho}{d\theta}\right|_j \approx \frac{h_{j-1}^2\rho_{j+1} - h_j^2\rho_{j-1} + \left(h_j^2 - h_{j-1}^2\right)\rho_j}{h_{j-1}h_j\,(h_{j-1}+h_j)}, \tag{13}$$

где $h_{j-1} = \theta_j - \theta_{j-1}$, $h_j = \theta_{j+1} - \theta_j$ — шаги сетки по углу.

На концах сетки используются односторонние разности первого порядка.

3.5 Алгоритм решения

Расчет выполняется в следующем порядке:

1) по таблицам вариантов формируются параметры частиц, функция и радиус экранирования, вычисляются a_Φ и E_r по формулам (3) и (5);

2) находятся $\rho_{\max}$ и сетка (11), для каждого узла сетки решается уравнение (7);

3) на логарифмической сетке расстояний от $R_{\min}/10$ до $2\rho_{\max}$ вычисляется потенциальная энергия (1);

4) на сетке углов $\theta_1 \in [0, \pi]$ из 361 точки вычисляются $k(\theta_1)$ и $E_1(\theta_1)$ по формуле (4);

5) для каждого узла сетки прицельных параметров вычисляется угол χ по формуле (12) и углы θ_1, θ_2 по формулам (8);

6) вычисляются сечения $\omega(\theta_1)$ и $\omega(\theta_2)$ по формулам (9) и (13);

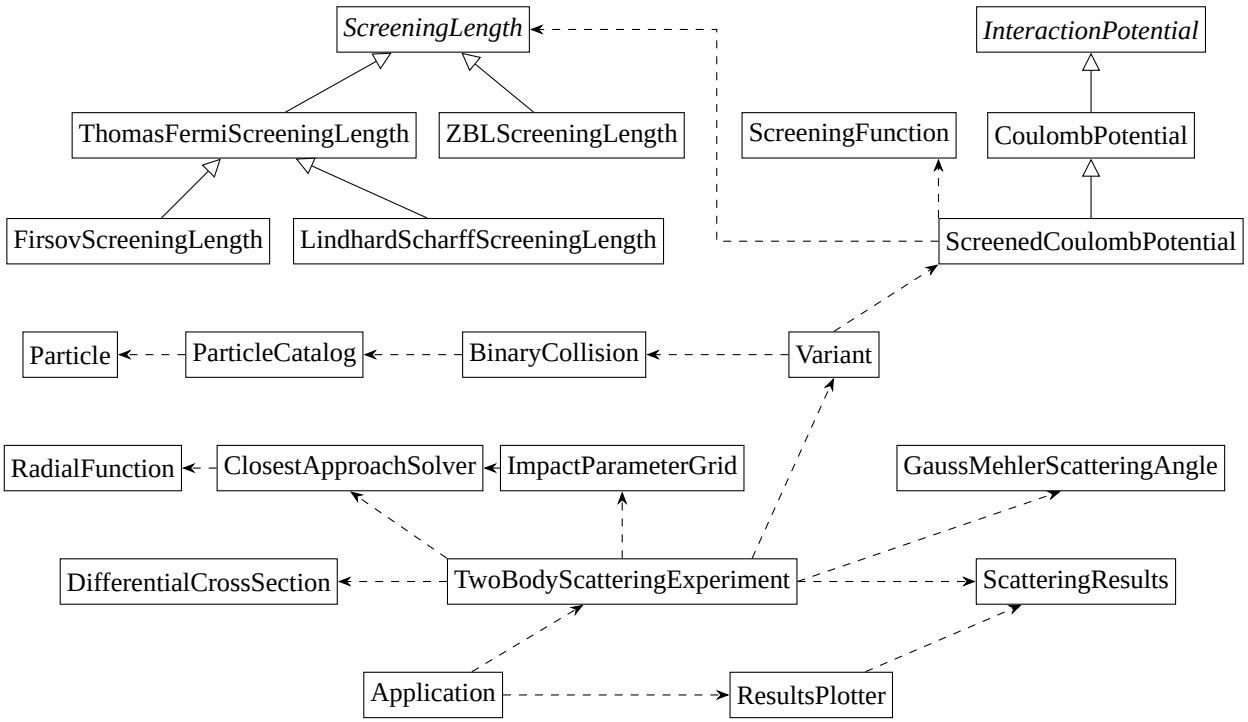
7) результаты выводятся в виде таблицы и графиков.

4 Программная реализация

Программа написана на языке Python 3 [5.] с использованием библиотек NumPy [6.] (векторные вычисления) и Matplotlib [7.] (построение графиков). Программа построена на принципах объектно-ориентированного программирования: каждая физическая сущность и каждый численный метод представлены отдельным классом, общие свойства вынесены в абстрактные базовые классы. Это позволяет менять потенциал, радиус экранирования или пару частиц без изменения алгоритма расчета. Назначение классов приведено в таблице 3, связи между ними показаны на рисунке 1. Текст программы приведен в приложении А.

Таблица 3 – Классы программы

Класс	Назначение
`Particle`, `ParticleCatalog`	Частица (заряд ядра, масса) и справочник частиц
`ScreeningFunction`	Функция экранирования (2); экземпляры `BOHR`, `MOLIERE`, `KR_C`, `ZBL`
`ScreeningLength` (абстрактный)	Радиус экранирования $a(Z_1, Z_2)$
`ThomasFermiScreeningLength`	Радиус экранирования модели Томаса – Ферми $a = 0{,}8853\, a_\text{Б}\, Z_{12}^{-1/3}$, $Z_{12} = (Z_1^x + Z_2^x)^y$
`FirsovScreeningLength`, `LindhardScharff ScreeningLength`, `ZBLScreeningLength`	Радиусы экранирования Фирсова, Линдхарда – Шарфа и ZBL
`InteractionPotential` (абстрактный)	Потенциальная энергия взаимодействия $U(r)$
`CoulombPotential`, `ScreenedCoulombPotential`	Кулоновский и экранированный кулоновский (1) потенциалы
`BinaryCollision`	Кинематика столкновения: E_r, $k(\theta_1)$, $E_1(\theta_1)$, пересчет χ в θ_1, θ_2
`RadialFunction`	Функция $g(r)$ из (6)
`ClosestApproachSolver`	Решение уравнения (7)
`ImpactParameterGrid`	Поиск ρ_max и построение сетки (11)
`GaussMehlerScatteringAngle`	Угол рассеяния по формуле (12)
`DifferentialCrossSection`	Сечение рассеяния (9)
`Variant`	Вариант задания и создание объектов задачи по таблицам вариантов
`TwoBodyScatteringExperiment`, `ScatteringResults`	Выполнение расчета и хранение результатов
`ResultsPlotter`	Построение графиков
`Application`	Консольный запуск программы



Курсивом обозначены абстрактные классы; сплошная линия с треугольником — наследование, штриховая линия со стрелкой — использование (ассоциация).

Рисунок 1 – Диаграмма классов программы

Программа запускается командой `python main.py` (по умолчанию выполняется расчет варианта 4, номер варианта задается параметром `--variant`). Результаты сохраняются в каталог `results/variant_04`: шесть графиков и таблица значений ρ, R, χ, θ_1, θ_2 в формате CSV.

5 Тестовые расчеты

Для проверки программы использовалось рассеяние в кулоновском поле ($F \equiv 1$), для которого известно точное решение [2.]:

$$R(\rho) = \frac{b}{2} + \sqrt{\frac{b^2}{4} + \rho^2}, \qquad \chi(\rho) = 2\,\mathrm{arctg}\,\frac{b}{2\rho}, \qquad \omega(\chi) = \left(\frac{b}{4}\right)^2 \frac{1}{\sin^4(\chi/2)}, \tag{14}$$

где $b = Z_1 Z_2 e^2 / E_r$ — расстояние наименьшего сближения при лобовом столкновении, нм.

Тестовый расчет выполнен для $Z_1 = 1$, $Z_2 = 29$, $E_r = 100$ эВ ($b = 0{,}4176$ нм) на 200 значениях прицельного параметра от $0{,}01\,b$ до $2\,b$. Получены следующие результаты:

– максимальная относительная погрешность расстояния наименьшего сближения не превышает 10^{-14};

– угол рассеяния, рассчитанный по формуле (12) при $N = 100$, совпадает с точным значением с погрешностью не более $3{,}2 \cdot 10^{-5}$ рад (рисунок 2, слева);

– относительная погрешность дифференциального сечения во внутренних узлах сетки не превышает 0,25 %, в крайних узлах, где используются односторонние разности, — 2 %.

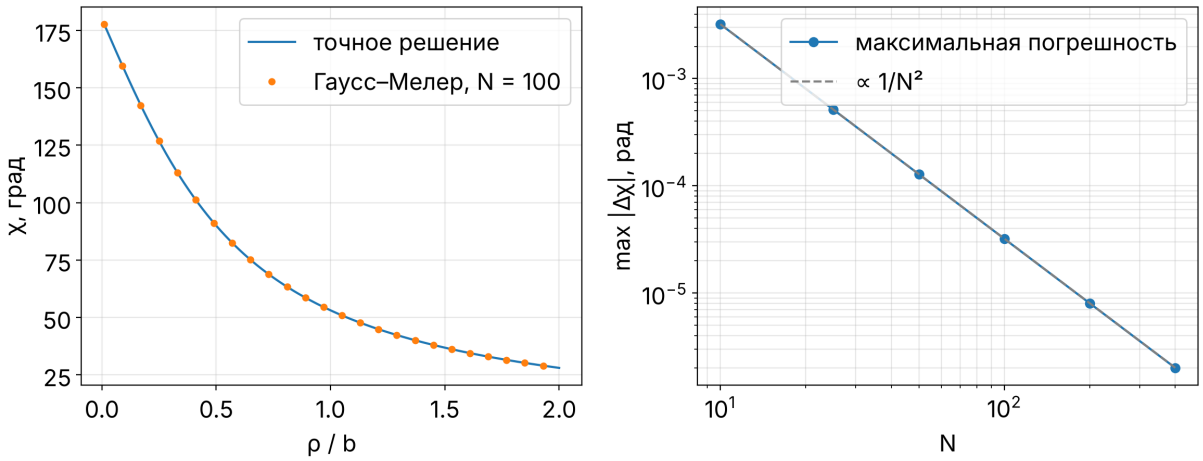


Рисунок 2 – Угол рассеяния в кулоновском поле (слева) и зависимость погрешности формулы Гаусса – Мелера от числа узлов (справа)

Зависимость погрешности угла рассеяния от числа узлов N приведена

в таблице 4. Для варианта 4 погрешность оценивалась по сравнению с расчетом при $N = 1600$. Погрешность убывает пропорционально $1/N^2$, что соответствует второму порядку точности формулы средних прямоугольников. При $N = 100$ погрешность угла рассеяния не превышает $5 \cdot 10^{-5}$ рад ($0{,}003°$), что достаточно для решаемой задачи.

Таблица 4 – Максимальная погрешность угла рассеяния χ, рад, в зависимости от числа узлов квадратуры

N	Кулоновский потенциал	Вариант 4 (Kr-C)
10	$3{,}22 \cdot 10^{-3}$	$4{,}12 \cdot 10^{-3}$
25	$5{,}14 \cdot 10^{-4}$	$6{,}57 \cdot 10^{-4}$
50	$1{,}28 \cdot 10^{-4}$	$1{,}64 \cdot 10^{-4}$
100	$3{,}21 \cdot 10^{-5}$	$4{,}09 \cdot 10^{-5}$
200	$8{,}03 \cdot 10^{-6}$	$1{,}01 \cdot 10^{-5}$
400	$2{,}01 \cdot 10^{-6}$	$2{,}41 \cdot 10^{-6}$

Кроме того, проверены предельные значения кинематического фактора $k(0) = 1$ и $k(\pi) = \left((m_2 - m_1)/(m_1 + m_2)\right)^2$, выполнение закона сохранения энергии $E_1 + E_2 = E_0$ при пересчете углов по формулам (8) и нормировка функций экранирования $F(0) = 1$. Все тесты (модуль `tests/test_two_body.py`, приложение А) выполняются успешно.

6 Результаты моделирования

6.1 Потенциальная энергия и кинематический фактор

Зависимость потенциальной энергии взаимодействия протона с атомом меди от расстояния приведена на рисунке 3. Горизонтальной линией отмечена энергия относительного движения $E_r = 0{,}0984$ эВ. Потенциальная энергия становится равной E_r на расстоянии $R(0) = 0{,}2774$ нм $\approx 20\,a_\Phi$; это расстояние наименьшего сближения при лобовом столкновении.

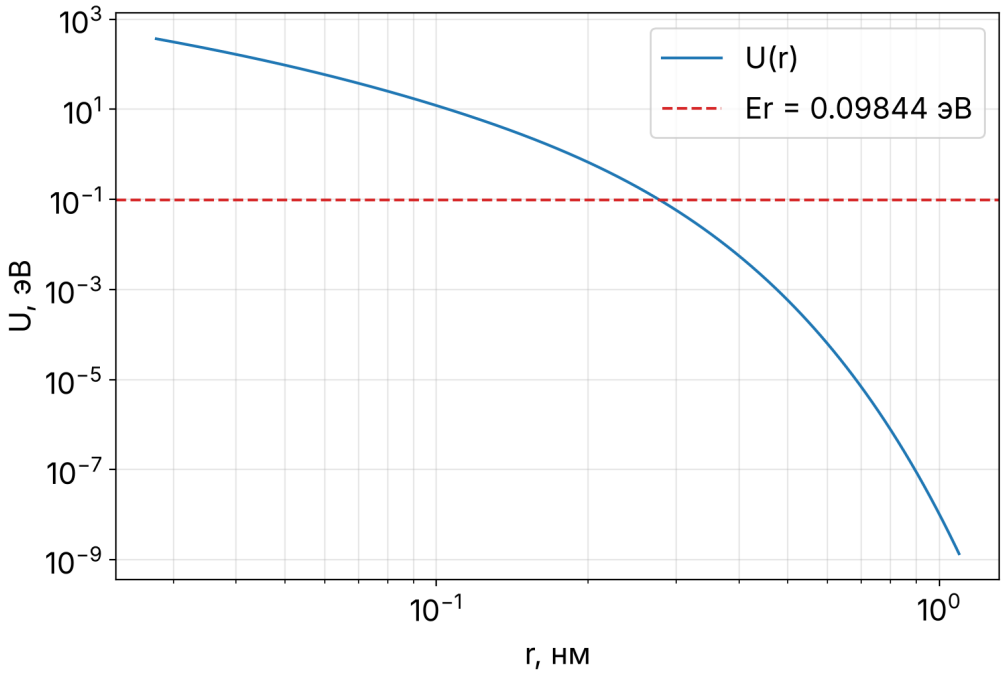


Рисунок 3 – Потенциальная энергия взаимодействия p – Cu (потенциал Kr-C, радиус экранирования Фирсова)

Зависимости кинематического фактора и энергии протона после рассеяния от угла рассеяния приведены на рисунке 4. Кинематический фактор убывает от 1 при $\theta_1 = 0$ до 0,9386 при $\theta_1 = 180°$; энергия рассеянного протона изменяется от 0,1 до 0,0939 эВ.

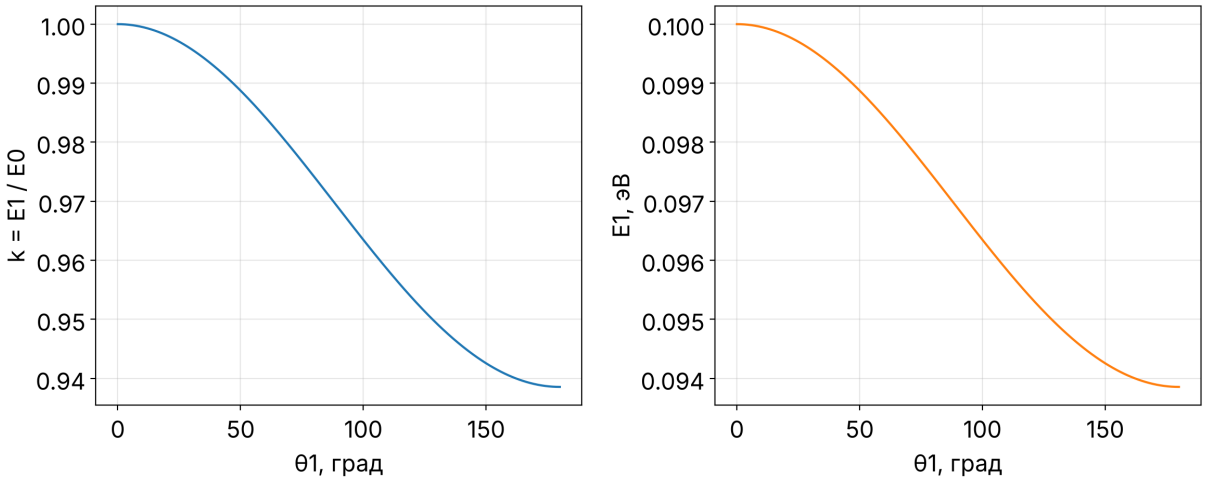


Рисунок 4 – Кинематический фактор $k(\theta_1)$ (слева) и энергия протона после рассеяния $E_1(\theta_1)$ (справа)

6.2 Сетка прицельных параметров

Максимальный прицельный параметр, удовлетворяющий условию (10), $\rho_{\max} = 0{,}54722$ нм, шаг сетки $\Delta\rho = 0{,}0027361$ нм. Зависимость расстояния наименьшего сближения от прицельного параметра приведена на рисунке 5.

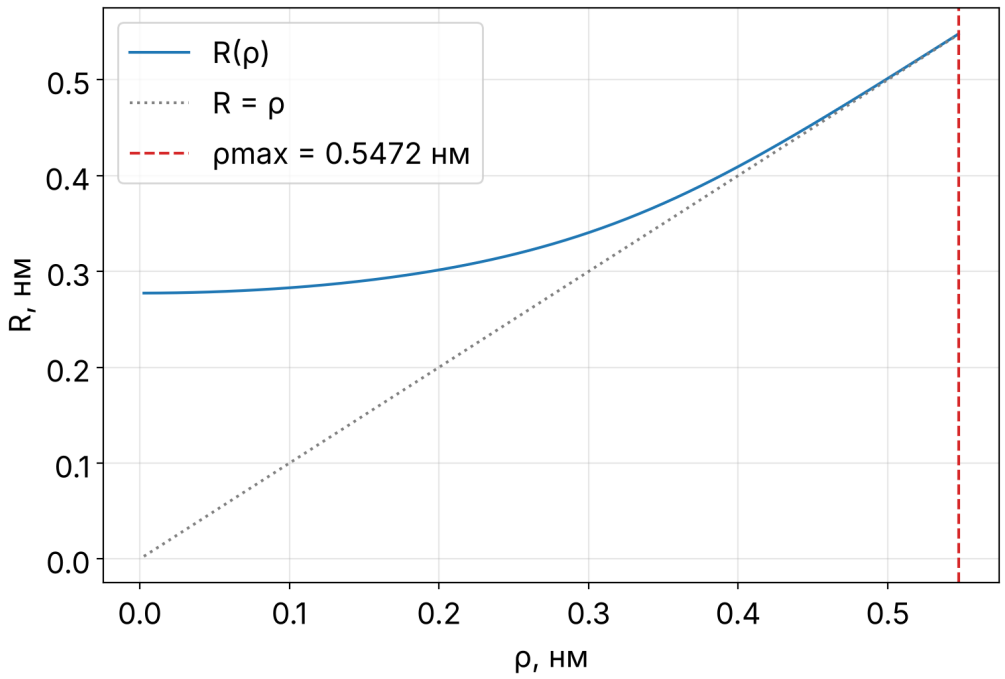


Рисунок 5 – Зависимость расстояния наименьшего сближения от прицельного параметра

6.3 Углы рассеяния

Зависимости угла рассеяния в ц-системе и углов рассеяния в л-системе от прицельного параметра приведены на рисунке 6 и в таблице 5 (приведен каждый двадцатый узел сетки).

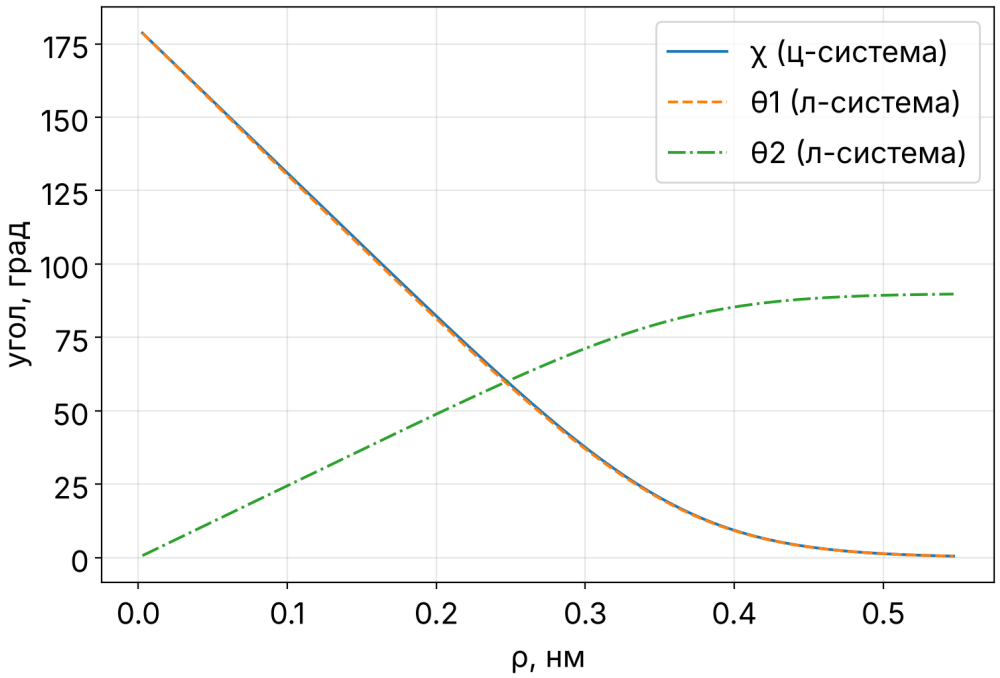


Рисунок 6 – Зависимости углов рассеяния χ, θ_1, θ_2 от прицельного параметра

Таблица 5 – Расстояние наименьшего сближения и углы рассеяния

ρ, нм	R, нм	χ, град	θ_1, град	θ_2, град
0,00274	0,27741	178,66	178,64	0,67
0,05472	0,27903	153,20	152,78	13,40
0,10944	0,28408	126,37	125,63	26,81
0,16417	0,29309	99,61	98,71	40,20
0,21889	0,30706	73,28	72,41	53,36
0,27361	0,32762	48,41	47,74	65,80
0,32833	0,35676	27,18	26,77	76,41
0,38305	0,39558	12,38	12,19	83,81
0,43777	0,44252	4,65	4,57	87,68
0,49250	0,49414	1,55	1,53	89,22
0,54722	0,54776	0,49	0,48	89,75

6.4 Дифференциальные сечения рассеяния

Зависимости прицельного параметра от углов рассеяния в л-системе приведены на рисунке 7, дифференциальные сечения упругого рассеяния — на рисунке 8.

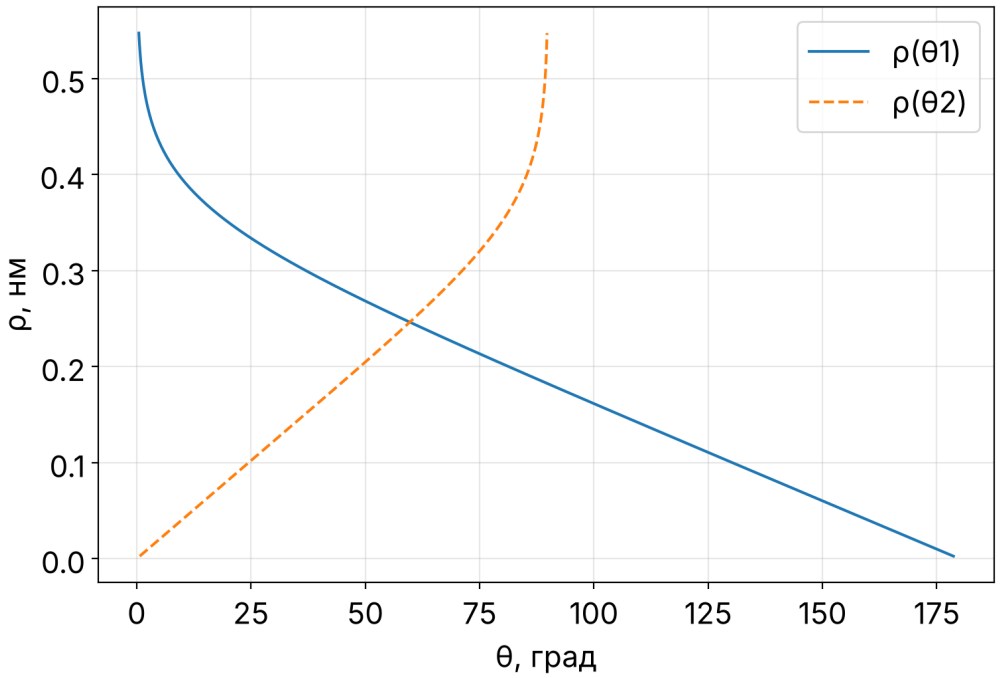


Рисунок 7 – Зависимости прицельного параметра от углов рассеяния θ_1 и θ_2

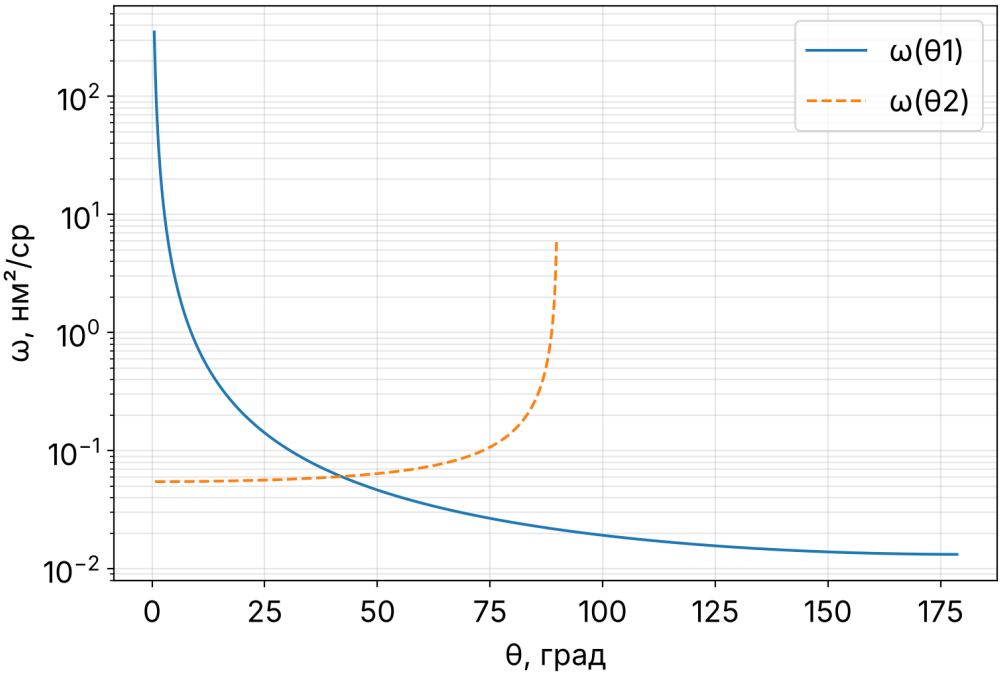


Рисунок 8 – Дифференциальные сечения упругого рассеяния $\omega(\theta_1)$ и $\omega(\theta_2)$

7 Анализ результатов

Масса атома меди в 63 раза больше массы протона, поэтому центр масс системы практически совпадает с атомом мишени: E_r отличается от E_0 на 1,6 %, угол рассеяния в л-системе θ_1 отличается от угла χ не более чем на $0{,}91^\circ$, а кинематический фактор близок к единице (протон теряет при упругом столкновении не более 6,1 % энергии). Угол вылета атома отдачи $\theta_2 = (\pi - \chi)/2$ не превышает 90°.

При энергии 0,1 эВ протон отражается на расстоянии около $20\,a_\Phi$ от ядра, где экранированный потенциал убывает практически экспоненциально: при уменьшении расстояния от 0,4 до 0,28 нм потенциальная энергия возрастает приблизительно в 17 раз (рисунок 3). Поэтому расстояние наименьшего сближения слабо зависит от прицельного параметра: при увеличении ρ от 0 до 0,2 нм оно возрастает лишь на 9 % (рисунок 5). Атом мишени ведет себя подобно жесткой сфере радиусом около 0,28 нм с “размытой” поверхностью: угол рассеяния $\chi(\rho)$ почти линейно убывает от 180° до 50° при росте ρ до 0,27 нм (рисунок 6).

При $\rho > 0{,}4$ нм частица пролетает в области, где $U(r) \ll E_r$, поэтому $R \to \rho$, а угол рассеяния быстро стремится к нулю (при $\rho = 0{,}44$ нм $\chi = 4{,}6^\circ$, при $\rho = \rho_{\max}$ $\chi = 0{,}49^\circ$). Это и определяет вид дифференциальных сечений (рисунок 8):

– сечение $\omega(\theta_1)$ резко возрастает при малых углах (0,76 нм^2/ср при $\theta_1 = 10^\circ$) и слабо зависит от угла при рассеянии назад (от 0,022 нм^2/ср при 90° до 0,013 нм^2/ср при 170°), что близко к сечению рассеяния на жесткой сфере $R(0)^2/4 = 0{,}019$ нм^2/ср;

– сечение $\omega(\theta_2)$ почти постоянно при малых углах вылета атома отдачи (0,055 нм^2/ср при $\theta_2 = 10^\circ$) и резко возрастает при $\theta_2 \to 90^\circ$, что соответствует скользящим столкновениям с малой передачей энергии.

Неограниченный рост сечений при $\theta_1 \to 0$ и $\theta_2 \to 90^\circ$ является особенностью классического описания рассеяния [1.] и обусловлен большими прицельными параметрами; в расчете он ограничен выбором $\rho_{\max}$.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы решена задача упругого рассеяния протона на атоме меди при энергии 0,1 эВ с экранированным кулоновским потенциалом Kr-C и радиусом экранирования Фирсова.

Разработана объектно-ориентированная программа на языке Python, в которой потенциалы, функции и радиусы экранирования, кинематика столкновения, численные методы и вариант задания представлены отдельными классами. Программа позволяет выполнить расчет для любого из 24 вариантов задания.

Корректность программы подтверждена сравнением с точным решением задачи рассеяния в кулоновском поле: погрешность угла рассеяния при 100 узлах квадратуры Гаусса – Мелера не превышает $3{,}2 \cdot 10^{-5}$ рад и убывает пропорционально $1/N^2$.

Получены следующие результаты:

– радиус экранирования Фирсова $a_{\Phi} = 0{,}013608$ нм, энергия в ц-системе $E_r = 0{,}098440$ эВ;

– расстояние наименьшего сближения при лобовом столкновении 0,2774 нм, максимальный прицельный параметр $\rho_{\max} = 0{,}5472$ нм;

– кинематический фактор изменяется от 1 до 0,9386;

– рассчитаны зависимости $\chi(\rho)$, $\theta_1(\rho)$, $\theta_2(\rho)$ и дифференциальные сечения $\omega(\theta_1)$, $\omega(\theta_2)$.

При малой энергии столкновения рассеяние протона на атоме меди близко к рассеянию на жесткой сфере радиусом около 0,28 нм: вероятность рассеяния на большие углы слабо зависит от угла, а при малых углах сечение резко возрастает.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Имитационное моделирование плазменных процессов : учебное пособие. — Москва : НИУ «МЭИ», 2025. — 110 с.

2. Ландау, Л. Д. Теоретическая физика : в 10 т. Т. 1. Механика / Л. Д. Ландау, Е. М. Лифшиц. — 5-е изд., стер. — Москва : Физматлит, 2004. — 224 с.

3. Экштайн, В. Компьютерное моделирование взаимодействия частиц с поверхностью твердого тела : пер. с англ. / В. Экштайн. — Москва : Мир, 1995. — 321 с.

4. Wilson, W. D. Calculations of nuclear stopping, ranges, and straggling in the low-energy region / W. D. Wilson, L. G. Haggmark, J. P. Biersack // Physical Review B. — 1977. — Vol. 15, no. 5. — P. 2458–2468.

5. Python 3 documentation : [сайт]. — URL: https://docs.python.org/3/ (дата обращения: 18.10.2026).

6. Harris, C. R. Array programming with NumPy / C. R. Harris, K. J. Millman, S. J. van der Walt [et al.] // Nature. — 2020. — Vol. 585. — P. 357–362.

7. Hunter, J. D. Matplotlib: A 2D graphics environment / J. D. Hunter // Computing in Science & Engineering. — 2007. — Vol. 9, no. 3. — P. 90–95.

ПРИЛОЖЕНИЕ А

(обязательное)

Текст программы

Программа состоит из пакета `two_body`, модуля запуска `main.py`, модуля тестов `tests/test_two_body.py` и модуля подготовки рисунков отчета `report/make_figures.py`.

А.1 Модуль particles.py

```
"""Частицы, участвующие в парных столкновениях."""

from dataclasses import dataclass


@dataclass(frozen=True)
class Particle:
    """Частица (ион или атом): обозначение, заряд ядра Z и масса в а.е.м."""

    symbol: str
    name: str
    z: int
    mass: float

    def __str__(self) -> str:
        return f"{self.symbol} (Z = {self.z}, M = {self.mass:g} а.е.м.)"


class ParticleCatalog:
    """Справочник частиц, встречающихся в вариантах задания."""

    _particles = {
        p.symbol: p
        for p in (
            Particle("p", "протон", 1, 1.007276),
            Particle("He", "гелий", 2, 4.002602),
            Particle("He2", "гелий (He2+)", 2, 4.002602),
            Particle("Ar", "аргон", 18, 39.948),
            Particle("Al", "алюминий", 13, 26.981539),
            Particle("Si", "кремний", 14, 28.0855),
            Particle("Cu", "медь", 29, 63.546),
            Particle("Nb", "ниобий", 41, 92.90637),
            Particle("Ag", "серебро", 47, 107.8682),
            Particle("Au", "золото", 79, 196.966569),
        )
    }

    @classmethod
    def get(cls, symbol: str) -> Particle:
        try:
            return cls._particles[symbol]
        except KeyError:
            raise KeyError(f"Неизвестная частица: {symbol!r}") from None
```

A.2 Модуль screening.py

```
"""Функции и длины экранирования для экранированных кулоновских потенциалов."""

from abc import ABC, abstractmethod

import numpy as np

A_BOHR = 0.0529  # первый боровский радиус, нм
THOMAS_FERMI_CONSTANT = (9.0 * np.pi**2 / 128.0) ** (1.0 / 3.0)  # 0,8853


class ScreeningFunction:
    """Функция экранирования F(x) = sum c_i * exp(-d_i * x), где x = r / a (табл. 1)."""

    def __init__(self, name: str, c, d):
        self.name = name
        self.c = np.asarray(c, dtype=float)
        self.d = np.asarray(d, dtype=float)
        if self.c.shape != self.d.shape:
            raise ValueError("Количество коэффициентов c_i и d_i должно совпадать")
        if not np.isclose(self.c.sum(), 1.0, atol=1e-5):
            raise ValueError(f"Сумма c_i должна быть равна 1, получено {self.c.sum()}")

    def __call__(self, x):
        x = np.asarray(x, dtype=float)
        return np.sum(self.c * np.exp(-np.multiply.outer(x, self.d)), axis=-1)

    def __repr__(self) -> str:
        return f"ScreeningFunction({self.name!r})"


BOHR = ScreeningFunction("Бора", [1.0], [1.0])
MOLIERE = ScreeningFunction("Мольера", [0.35, 0.55, 0.10], [0.3, 1.2, 6.0])
KR_C = ScreeningFunction(
    "Kr-C", [0.190945, 0.473674, 0.335381], [0.278544, 0.637174, 1.919249]
)
# В табл. 1 пособия коэффициенты ZBL дают sum c_i = 0,945 (F(0) != 1),
# поэтому используются стандартные коэффициенты Циглера–Бирзака–Литтмарка.
ZBL = ScreeningFunction(
    "ZBL", [0.02817, 0.28022, 0.50986, 0.18175], [0.20162, 0.40290, 0.94229, 3.19980]
)


class ScreeningLength(ABC):
    """Длина экранирования a(Z1, Z2), нм."""

    name = ""

    @abstractmethod
    def __call__(self, z1: int, z2: int) -> float:
        ...

    def __repr__(self) -> str:
        return f"{type(self).__name__}()"


class ThomasFermiScreeningLength(ScreeningLength):
```

```
    """a = 0,8853 * a_B * Z12^(-1/3), где Z12 = (Z1^x + Z2^x)^y и x*y = 1."""

    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def effective_charge(self, z1: int, z2: int) -> float:
        return (z1**self.x + z2**self.x) ** self.y

    def __call__(self, z1: int, z2: int) -> float:
        return THOMAS_FERMI_CONSTANT * A_BOHR * self.effective_charge(z1, z2) ** (-1.0
        ↪  / 3.0)


class FirsovScreeningLength(ThomasFermiScreeningLength):
    """Радиус экранирования Фирсова: a = 0,8853 a_B (Z1^1/2 + Z2^1/2)^(-2/3)."""

    name = "Фирсова"

    def __init__(self):
        super().__init__(x=1.0 / 2.0, y=2.0)


class LindhardScharffScreeningLength(ThomasFermiScreeningLength):
    """Радиус экранирования Линдхарда–Шарфа: a = 0,8853 a_B (Z1^2/3 +
    ↪  Z2^2/3)^(-1/2)."""

    name = "Линдхарда-Шарфа"

    def __init__(self):
        super().__init__(x=2.0 / 3.0, y=3.0 / 2.0)


class ZBLScreeningLength(ScreeningLength):
    """Универсальная длина экранирования: a_U = 0,8853 a_B (Z1^0,23 +
    ↪  Z2^0,23)^(-1)."""

    name = "ZBL"

    def __call__(self, z1: int, z2: int) -> float:
        return THOMAS_FERMI_CONSTANT * A_BOHR / (z1**0.23 + z2**0.23)
```

A.3 Модуль potentials.py

```
"""Потенциалы взаимодействия двух частиц."""

from abc import ABC, abstractmethod

import numpy as np

from .screening import ScreeningFunction, ScreeningLength

E2 = 1.44  # квадрат элементарного заряда e^2, эВ*нм


class InteractionPotential(ABC):
    """Потенциальная энергия взаимодействия U(r), эВ (r в нм)."""
```

```
14
15      def __init__(self, z1: int, z2: int):
16          self.z1 = z1
17          self.z2 = z2
18
19      @property
20      @abstractmethod
21      def name(self) -> str:
22          ...
23
24      @abstractmethod
25      def __call__(self, r):
26          ...
27
28
29  class CoulombPotential(InteractionPotential):
30      """Кулоновский потенциал (6): U(r) = Z1 Z2 e^2 / r."""
31
32      @property
33      def name(self) -> str:
34          return "Кулоновский"
35
36      def __call__(self, r):
37          return self.z1 * self.z2 * E2 / np.asarray(r, dtype=float)
38
39
40  class ScreenedCoulombPotential(CoulombPotential):
41      """Экранированный кулоновский потенциал (7): U(r) = Z1 Z2 e^2 / r * F(r / a)."""
42
43      def __init__(
44          self,
45          z1: int,
46          z2: int,
47          screening_function: ScreeningFunction,
48          screening_length: ScreeningLength,
49      ):
50          super().__init__(z1, z2)
51          self.screening_function = screening_function
52          self.screening_length = screening_length
53          self.a = screening_length(z1, z2)
54
55      @property
56      def name(self) -> str:
57          return (
58              f"{self.screening_function.name} "
59              f"(радиус экранирования {self.screening_length.name})"
60          )
61
62      def __call__(self, r):
63          r = np.asarray(r, dtype=float)
64          return super().__call__(r) * self.screening_function(r / self.a)
```

A.4 Модуль kinematics.py

```
1   """Кинематика упругого столкновения налетающей частицы с покоящейся."""
2
3   import numpy as np
```

```python

from .particles import Particle


class BinaryCollision:
    """Парное упругое столкновение: налетающая частица m1 с энергией E0 и покоящаяся m2."""

    def __init__(self, projectile: Particle, target: Particle, energy: float):
        if energy <= 0:
            raise ValueError("Энергия налетающей частицы должна быть положительной")
        self.projectile = projectile
        self.target = target
        self.energy = energy  # E0, эВ

    @property
    def m1(self) -> float:
        return self.projectile.mass

    @property
    def m2(self) -> float:
        return self.target.mass

    @property
    def relative_energy(self) -> float:
        """Энергия в системе центра масс: Er = E0 * m2 / (m1 + m2)."""
        return self.energy * self.m2 / (self.m1 + self.m2)

    @property
    def max_lab_angle(self) -> float:
        """Максимальный угол рассеяния налетающей частицы в л-системе."""
        if self.m1 <= self.m2:
            return np.pi
        return float(np.arcsin(self.m2 / self.m1))

    def kinematic_factor(self, theta1):
        """Кинематический фактор (15): k = E1 / E0 как функция угла θ1.

        Для m1 > m2 возвращается ветвь со знаком «+», за пределами
        максимального угла рассеяния — NaN.
        """
        theta1 = np.asarray(theta1, dtype=float)
        ratio = self.m2 / self.m1
        discriminant = ratio**2 - np.sin(theta1) ** 2
        root = np.sqrt(np.where(discriminant >= 0, discriminant, np.nan))
        return (self.m1 / (self.m1 + self.m2)) ** 2 * (np.cos(theta1) + root) ** 2

    def scattered_energy(self, theta1):
        """Энергия налетающей частицы после рассеяния E1 = k * E0, эВ."""
        return self.kinematic_factor(theta1) * self.energy

    def lab_angles(self, chi):
        """Углы рассеяния в л-системе (17) по углу χ в ц-системе: (θ1, θ2)."""
        chi = np.asarray(chi, dtype=float)
        theta1 = np.arctan2(self.m2 * np.sin(chi), self.m1 + self.m2 * np.cos(chi))
        theta2 = (np.pi - chi) / 2.0
        return theta1, theta2
```

A.5 Модуль scattering.py

```
"""Расчёт расстояния наименьшего сближения, сетки прицельных параметров
и угла рассеяния в системе центра масс."""

import numpy as np

from .potentials import InteractionPotential


class RadialFunction:
    """g(r) = 1 - U(r) / Er - ρ^2 / r^2 — подкоренное выражение интеграла рассеяния
    ↪  (16)."""

    def __init__(self, potential: InteractionPotential, relative_energy: float):
        self.potential = potential
        self.relative_energy = relative_energy

    def __call__(self, r, rho: float):
        r = np.asarray(r, dtype=float)
        return 1.0 - self.potential(r) / self.relative_energy - (rho / r) ** 2


class ClosestApproachSolver:
    """Решает уравнение g(R) = 0 относительно расстояния наименьшего сближения R
    методом деления отрезка пополам (для отталкивающего потенциала g(r) монотонно
    возрастает, поэтому корень единственный)."""

    def __init__(self, g: RadialFunction, rel_tol: float = 1e-14, max_iter: int =
    ↪  200):
        self.g = g
        self.rel_tol = rel_tol
        self.max_iter = max_iter

    def __call__(self, rho: float) -> float:
        hi = max(rho, 1e-3)
        while self.g(hi, rho) < 0:
            hi *= 2.0
        lo = hi
        while self.g(lo, rho) >= 0:
            lo /= 2.0
        for _ in range(self.max_iter):
            mid = 0.5 * (lo + hi)
            if self.g(mid, rho) < 0:
                lo = mid
            else:
                hi = mid
            if hi - lo <= self.rel_tol * hi:
                break
        # Возвращаем правую границу: на ней g(R) >= 0, что гарантирует
        # неотрицательность подкоренного выражения при r > R.
        return hi

    def solve_many(self, rho) -> np.ndarray:
        return np.array([self(value) for value in np.atleast_1d(rho)])


class ImpactParameterGrid:
    """Сетка прицельных параметров ρ в интервале [ρmax/n, ρmax] с шагом ρmax/n,
    где ρmax — наименьший ρ, для которого |1 - R/ρ| < tolerance."""
```

```
57
58      def __init__(
59          self,
60          solver: ClosestApproachSolver,
61          tolerance: float = 1e-3,
62          n_points: int = 200,
63      ):
64          self.solver = solver
65          self.tolerance = tolerance
66          self.n_points = n_points
67
68      def _deviation(self, rho: float) -> float:
69          return abs(1.0 - self.solver(rho) / rho)
70
71      def find_rho_max(self) -> float:
72          hi = self.solver(0.0)  # расстояние сближения при лобовом ударе
73          while self._deviation(hi) >= self.tolerance:
74              hi *= 2.0
75          lo = hi / 2.0
76          if self._deviation(lo) < self.tolerance:
77              return lo
78          for _ in range(100):
79              mid = 0.5 * (lo + hi)
80              if self._deviation(mid) < self.tolerance:
81                  hi = mid
82              else:
83                  lo = mid
84              if hi - lo <= 1e-12 * hi:
85                  break
86          return hi
87
88      def build(self) -> np.ndarray:
89          rho_max = self.find_rho_max()
90          step = rho_max / self.n_points
91          return step * np.arange(1, self.n_points + 1)
92
93
94  class GaussMehlerScatteringAngle:
95      """Угол рассеяния в ц-системе χ(ρ) по квадратурной формуле Гаусса–Мелера:
96
97          χ = π - 2 (ρ/R) (π/N) Σ ξ_n ξ_{N-n+1} / sqrt(g(R / ξ_n^2)),
98          ξ_n = cos((2n - 1) π / (4N)).
99      """
100
101     def __init__(self, g: RadialFunction, n_nodes: int = 100):
102         self.g = g
103         self.n_nodes = n_nodes
104         n = np.arange(1, n_nodes + 1)
105         self.nodes = np.cos((2 * n - 1) * np.pi / (4 * n_nodes))
106         self.weights = self.nodes * self.nodes[::-1]
107
108     def __call__(self, rho: float, closest_approach: float) -> float:
109         r = closest_approach / self.nodes**2
110         integrand = self.weights / np.sqrt(self.g(r, rho))
111         return np.pi - 2.0 * rho / closest_approach * np.pi / self.n_nodes *
            ↪  integrand.sum()
112
113     def compute_many(self, rho, closest_approach) -> np.ndarray:
114         return np.array([self(p, r) for p, r in zip(rho, closest_approach)])
```

A.6 Модуль cross_section.py

```
"""Дифференциальное сечение упругого рассеяния."""

import numpy as np


class DifferentialCrossSection:
    """Сечение (19): ω(θ) = ρ(θ) / sin θ * |dρ/dθ|, нм^2/ср.

    Зависимость ρ(θ) задаётся таблично (обратная к θ(ρ)), производная
    находится численно по неравномерной сетке.
    """

    def __init__(self, rho, theta):
        rho = np.asarray(rho, dtype=float)
        theta = np.asarray(theta, dtype=float)
        order = np.argsort(theta)
        self.theta = theta[order]
        self.rho = rho[order]
        self.omega = self.rho / np.sin(self.theta) * np.abs(np.gradient(self.rho,
        ↪  self.theta))
```

A.7 Модуль variants.py

```
"""Варианты расчётного задания практической работы № 1."""

from dataclasses import dataclass

from .kinematics import BinaryCollision
from .particles import ParticleCatalog
from .potentials import ScreenedCoulombPotential
from .screening import (
    BOHR,
    KR_C,
    MOLIERE,
    ZBL,
    FirsovScreeningLength,
    LindhardScharffScreeningLength,
    ScreeningFunction,
    ScreeningLength,
    ZBLScreeningLength,
)


@dataclass(frozen=True)
class PotentialOption:
    """Столбец таблицы «А»: потенциал и радиус экранирования."""

    screening_function: ScreeningFunction
    screening_length_type: type


@dataclass(frozen=True)
class CollisionOption:
    """Столбец таблицы «Б»: энергия и пара сталкивающихся частиц."""
```

```
32
33      energy: float  # E0, эВ
34      projectile: str
35      target: str
36
37
38  POTENTIAL_OPTIONS = {
39      1: PotentialOption(BOHR, FirsovScreeningLength),
40      2: PotentialOption(MOLIERE, LindhardScharffScreeningLength),
41      3: PotentialOption(KR_C, FirsovScreeningLength),
42      4: PotentialOption(ZBL, ZBLScreeningLength),
43  }
44
45  COLLISION_OPTIONS = {
46      1: CollisionOption(0.1, "p", "Cu"),
47      2: CollisionOption(0.5, "He2", "Au"),
48      3: CollisionOption(0.3, "He", "Al"),
49      4: CollisionOption(0.8, "p", "Si"),
50      5: CollisionOption(1.0, "Ar", "Ag"),
51      6: CollisionOption(1.5, "p", "Nb"),
52  }
53
54  # № варианта -> (A, Б)
55  VARIANT_TABLE = {
56      1: (1, 6), 2: (4, 5), 3: (4, 2), 4: (3, 1), 5: (4, 6), 6: (1, 2),
57      7: (2, 5), 8: (2, 3), 9: (4, 4), 10: (1, 1), 11: (3, 6), 12: (3, 5),
58      13: (4, 1), 14: (1, 5), 15: (2, 4), 16: (2, 3), 17: (3, 4), 18: (2, 6),
59      19: (4, 3), 20: (2, 1), 21: (2, 2), 22: (3, 2), 23: (3, 3), 24: (1, 4),
60  }
61
62
63  @dataclass(frozen=True)
64  class Variant:
65      """Индивидуальный вариант задания."""
66
67      number: int
68      potential_index: int
69      collision_index: int
70
71      @classmethod
72      def from_number(cls, number: int) -> "Variant":
73          if number not in VARIANT_TABLE:
74              raise ValueError(f"Вариант должен быть от 1 до {len(VARIANT_TABLE)}")
75          return cls(number, *VARIANT_TABLE[number])
76
77      @property
78      def potential_option(self) -> PotentialOption:
79          return POTENTIAL_OPTIONS[self.potential_index]
80
81      @property
82      def collision_option(self) -> CollisionOption:
83          return COLLISION_OPTIONS[self.collision_index]
84
85      def build_collision(self) -> BinaryCollision:
86          option = self.collision_option
87          return BinaryCollision(
88              ParticleCatalog.get(option.projectile),
89              ParticleCatalog.get(option.target),
90              option.energy,
91          )
```

```
92
93      def build_potential(self) -> ScreenedCoulombPotential:
94          collision = self.build_collision()
95          option = self.potential_option
96          screening_length: ScreeningLength = option.screening_length_type()
97          return ScreenedCoulombPotential(
98              collision.projectile.z,
99              collision.target.z,
100             option.screening_function,
101             screening_length,
102         )
103
104     def describe(self) -> str:
105         collision = self.build_collision()
106         potential = self.build_potential()
107         return "\n".join(
108             [
109                 f"Вариант {self.number}: A = {self.potential_index}, Б =
                    ↪ {self.collision_index}",
110                 f"  Потенциал: {potential.name}",
111                 f"  Налетающая частица: {collision.projectile}",
112                 f"  Покоящаяся частица: {collision.target}",
113                 f"  E0 = {collision.energy:g} эВ, Er = {collision.relative_energy:.6g}
                    ↪ эВ",
114                 f"  Длина экранирования a = {potential.a:.6g} нм",
115             ]
116         )
```

A.8 Модуль experiment.py

```
1  """Вычислительный эксперимент: последовательное выполнение пп. 1–4 задания."""
2
3  import csv
4  from dataclasses import dataclass
5  from pathlib import Path
6
7  import numpy as np
8
9  from .cross_section import DifferentialCrossSection
10 from .kinematics import BinaryCollision
11 from .potentials import InteractionPotential
12 from .scattering import (
13     ClosestApproachSolver,
14     GaussMehlerScatteringAngle,
15     ImpactParameterGrid,
16     RadialFunction,
17 )
18 from .variants import Variant
19
20
21 @dataclass
22 class ScatteringResults:
23     """Результаты расчёта для одного варианта."""
24
25     variant: Variant
26     collision: BinaryCollision
27     potential: InteractionPotential
```

```
    # п. 1а: потенциальная энергия
    r: np.ndarray
    potential_energy: np.ndarray
    # п. 1б: кинематический фактор и энергия после рассеяния
    theta1_grid: np.ndarray
    kinematic_factor: np.ndarray
    scattered_energy: np.ndarray
    # п. 2: сетка прицельных параметров и расстояние наименьшего сближения
    rho_max: float
    rho: np.ndarray
    closest_approach: np.ndarray
    # п. 3: углы рассеяния
    chi: np.ndarray
    theta1: np.ndarray
    theta2: np.ndarray
    # п. 4: дифференциальные сечения
    cross_section1: DifferentialCrossSection
    cross_section2: DifferentialCrossSection

    def save_csv(self, path: Path) -> None:
        """Таблица ρ, R, χ, θ1, θ2 (углы в градусах) для отчёта."""
        with open(path, "w", newline="", encoding="utf-8") as f:
            writer = csv.writer(f)
            writer.writerow(["rho_nm", "R_nm", "chi_deg", "theta1_deg", "theta2_deg"])
            for row in zip(
                self.rho,
                self.closest_approach,
                np.degrees(self.chi),
                np.degrees(self.theta1),
                np.degrees(self.theta2),
            ):
                writer.writerow([f"{value:.8g}" for value in row])


class TwoBodyScatteringExperiment:
    """Решение задачи двух тел для заданного варианта."""

    def __init__(
        self,
        variant: Variant,
        n_nodes: int = 100,
        n_rho: int = 200,
        rho_tolerance: float = 1e-3,
        n_angles: int = 361,
    ):
        self.variant = variant
        self.collision = variant.build_collision()
        self.potential = variant.build_potential()
        self.g = RadialFunction(self.potential, self.collision.relative_energy)
        self.solver = ClosestApproachSolver(self.g)
        self.grid = ImpactParameterGrid(self.solver, rho_tolerance, n_rho)
        self.angle = GaussMehlerScatteringAngle(self.g, n_nodes)
        self.n_angles = n_angles

    def run(self) -> ScatteringResults:
        # п. 2: сетка прицельных параметров и R(ρ)
        rho = self.grid.build()
        closest_approach = self.solver.solve_many(rho)

        # п. 1а: U(r) в интервале, охватывающем все расстояния сближения
```

```
        r = np.geomspace(closest_approach.min() / 10.0, rho[-1] * 2.0, 500)
        potential_energy = self.potential(r)

        # п. 1б: k(θ1) и E1(θ1)
        theta1_grid = np.linspace(0.0, self.collision.max_lab_angle, self.n_angles)
        kinematic_factor = self.collision.kinematic_factor(theta1_grid)
        scattered_energy = kinematic_factor * self.collision.energy

        # п. 3: χ(ρ), θ1(ρ), θ2(ρ)
        chi = self.angle.compute_many(rho, closest_approach)
        theta1, theta2 = self.collision.lab_angles(chi)

        # п. 4: ρ(θ) и ω(θ)
        cross_section1 = DifferentialCrossSection(rho, theta1)
        cross_section2 = DifferentialCrossSection(rho, theta2)

        return ScatteringResults(
            variant=self.variant,
            collision=self.collision,
            potential=self.potential,
            r=r,
            potential_energy=potential_energy,
            theta1_grid=theta1_grid,
            kinematic_factor=kinematic_factor,
            scattered_energy=scattered_energy,
            rho_max=float(rho[-1]),
            rho=rho,
            closest_approach=closest_approach,
            chi=chi,
            theta1=theta1,
            theta2=theta2,
            cross_section1=cross_section1,
            cross_section2=cross_section2,
        )
```

A.9 Модуль plotting.py

```
"""Построение графиков по результатам расчёта."""

from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
import numpy as np  # noqa: E402

from .experiment import ScatteringResults  # noqa: E402


class ResultsPlotter:
    """Строит и сохраняет графики пп. 1–4 задания."""

    def __init__(
        self,
        results: ScatteringResults,
        output_dir: Path,
```

```
        dpi: int = 150,
        show_titles: bool = True,
    ):
        self.results = results
        self.output_dir = Path(output_dir)
        self.dpi = dpi
        # Без заголовков графики вставляются в отчёт: название рисунка — в подписи
        self.show_titles = show_titles
        collision = results.collision
        self.subtitle = (
            f"{collision.projectile.symbol} → {collision.target.symbol}, "
            f"E0 = {collision.energy:g} эВ, {results.potential.name}"
        )

    def _new_figure(self, title: str):
        fig, ax = plt.subplots(figsize=(8, 5.5))
        if self.show_titles:
            ax.set_title(f"{title}\n{self.subtitle}", fontsize=11)
        ax.grid(True, which="both", alpha=0.3)
        return fig, ax

    def _save(self, fig, name: str) -> Path:
        path = self.output_dir / name
        fig.tight_layout()
        fig.savefig(path, dpi=self.dpi)
        plt.close(fig)
        return path

    def plot_potential(self) -> Path:
        res = self.results
        fig, ax = self._new_figure("Потенциальная энергия взаимодействия U(r)")
        ax.loglog(res.r, res.potential_energy, label="U(r)")
        ax.axhline(res.collision.relative_energy, color="tab:red", ls="--",
                   label=f"Er = {res.collision.relative_energy:.4g} эВ")
        ax.set_xlabel("r, нм")
        ax.set_ylabel("U, эВ")
        ax.legend()
        return self._save(fig, "1a_potential.png")

    def plot_kinematic_factor(self) -> Path:
        res = self.results
        fig, (ax_k, ax_e) = plt.subplots(1, 2, figsize=(12, 5))
        if self.show_titles:
            fig.suptitle(f"Кинематический фактор k(θ1) и энергия
            ↪  E1(θ1)\n{self.subtitle}",
                         fontsize=11)
        angles = np.degrees(res.theta1_grid)
        ax_k.plot(angles, res.kinematic_factor, color="tab:blue")
        ax_k.set_ylabel("k = E1 / E0")
        ax_e.plot(angles, res.scattered_energy, color="tab:orange")
        ax_e.set_ylabel("E1, эВ")
        for ax in (ax_k, ax_e):
            ax.set_xlabel("θ1, град")
            ax.grid(True, alpha=0.3)
        return self._save(fig, "1b_kinematic_factor.png")

    def plot_closest_approach(self) -> Path:
        res = self.results
        fig, ax = self._new_figure("Расстояние наименьшего сближения R(ρ)")
        ax.plot(res.rho, res.closest_approach, label="R(ρ)")
```

```
        ax.plot(res.rho, res.rho, color="gray", ls=":", label="R = ρ")
        ax.axvline(res.rho_max, color="tab:red", ls="--",
                   label=f"ρmax = {res.rho_max:.4g} нм")
        ax.set_xlabel("ρ, нм")
        ax.set_ylabel("R, нм")
        ax.legend()
        return self._save(fig, "2_closest_approach.png")

    def plot_scattering_angles(self) -> Path:
        res = self.results
        fig, ax = self._new_figure("Углы рассеяния χ(ρ), θ1(ρ), θ2(ρ)")
        ax.plot(res.rho, np.degrees(res.chi), label="χ (ц-система)")
        ax.plot(res.rho, np.degrees(res.theta1), ls="--", label="θ1 (л-система)")
        ax.plot(res.rho, np.degrees(res.theta2), ls="-.", label="θ2 (л-система)")
        ax.set_xlabel("ρ, нм")
        ax.set_ylabel("угол, град")
        ax.legend()
        return self._save(fig, "3_scattering_angles.png")

    def plot_impact_parameter(self) -> Path:
        res = self.results
        fig, ax = self._new_figure("Прицельный параметр ρ(θ1) и ρ(θ2)")
        cs1, cs2 = res.cross_section1, res.cross_section2
        ax.plot(np.degrees(cs1.theta), cs1.rho, label="ρ(θ1)")
        ax.plot(np.degrees(cs2.theta), cs2.rho, ls="--", label="ρ(θ2)")
        ax.set_xlabel("θ, град")
        ax.set_ylabel("ρ, нм")
        ax.legend()
        return self._save(fig, "4a_impact_parameter.png")

    def plot_cross_sections(self) -> Path:
        res = self.results
        fig, ax = self._new_figure("Дифференциальное сечение упругого рассеяния ω(θ)")
        cs1, cs2 = res.cross_section1, res.cross_section2
        ax.semilogy(np.degrees(cs1.theta), cs1.omega, label="ω(θ1)")
        ax.semilogy(np.degrees(cs2.theta), cs2.omega, ls="--", label="ω(θ2)")
        ax.set_xlabel("θ, град")
        ax.set_ylabel("ω, нм²/ср")
        ax.legend()
        return self._save(fig, "4b_cross_sections.png")

    def plot_all(self) -> list:
        self.output_dir.mkdir(parents=True, exist_ok=True)
        return [
            self.plot_potential(),
            self.plot_kinematic_factor(),
            self.plot_closest_approach(),
            self.plot_scattering_angles(),
            self.plot_impact_parameter(),
            self.plot_cross_sections(),
        ]
```

A.10 Модуль __init__.py

```
"""Практическое задание № 1. Решение задачи двух тел."""

from .experiment import ScatteringResults, TwoBodyScatteringExperiment
```

```
from .variants import Variant

__all__ = ["ScatteringResults", "TwoBodyScatteringExperiment", "Variant"]
```

A.11 Модуль main.py

```
"""Запуск расчёта практического задания № 1 (по умолчанию — вариант 4)."""

import argparse
from pathlib import Path

import numpy as np

from two_body import TwoBodyScatteringExperiment, Variant
from two_body.plotting import ResultsPlotter


class Application:
    """Консольное приложение: расчёт, вывод сводки, сохранение графиков и таблицы."""

    def __init__(self, args: argparse.Namespace):
        self.variant = Variant.from_number(args.variant)
        self.output_dir = Path(args.output) / f"variant_{self.variant.number:02d}"
        self.experiment = TwoBodyScatteringExperiment(
            self.variant, n_nodes=args.nodes, n_rho=args.points
        )

    @staticmethod
    def parse_args() -> argparse.Namespace:
        parser = argparse.ArgumentParser(description=__doc__)
        parser.add_argument("--variant", type=int, default=4, help="номер варианта
        ↪  (1–24)")
        parser.add_argument("--nodes", type=int, default=100,
                            help="число узлов квадратуры Гаусса–Мелера")
        parser.add_argument("--points", type=int, default=200,
                            help="число точек сетки прицельных параметров")
        parser.add_argument("--output", default=str(Path(__file__).parent /
        ↪  "results"),
                            help="каталог для результатов")
        return parser.parse_args()

    def run(self) -> None:
        print(self.variant.describe())
        results = self.experiment.run()

        print(f"\nρmax = {results.rho_max:.6g} нм, "
              f"Δρ = {results.rho_max / len(results.rho):.6g} нм")
        print(f"R(ρ → 0) = {self.experiment.solver(0.0):.6g} нм")
        print(f"\n{'ρ, нм':>12} {'R, нм':>12} {'χ, °':>10} {'θ1, °':>10} {'θ2,
        ↪  °':>10}")
        for i in np.linspace(0, len(results.rho) - 1, 11, dtype=int):
            print(f"{results.rho[i]:12.6f} {results.closest_approach[i]:12.6f} "
                  f"{np.degrees(results.chi[i]):10.4f}
                  ↪  {np.degrees(results.theta1[i]):10.4f} "
                  f"{np.degrees(results.theta2[i]):10.4f}")

        self.output_dir.mkdir(parents=True, exist_ok=True)
```

```
        results.save_csv(self.output_dir / "scattering_table.csv")
        paths = ResultsPlotter(results, self.output_dir).plot_all()
        print(f"\nРезультаты сохранены в {self.output_dir}:")
        for path in paths:
            print(f"  {path.name}")


if __name__ == "__main__":
    Application(Application.parse_args()).run()
```

A.12 Модуль tests/test_two_body.py

```
"""Тестовые расчёты: сравнение численных результатов с аналитическими."""

import unittest

import numpy as np

from two_body import TwoBodyScatteringExperiment, Variant
from two_body.cross_section import DifferentialCrossSection
from two_body.kinematics import BinaryCollision
from two_body.particles import ParticleCatalog
from two_body.potentials import E2, CoulombPotential
from two_body.scattering import (
    ClosestApproachSolver,
    GaussMehlerScatteringAngle,
    RadialFunction,
)
from two_body.screening import (
    A_BOHR,
    BOHR,
    KR_C,
    MOLIERE,
    ZBL,
    FirsovScreeningLength,
    LindhardScharffScreeningLength,
)


class CoulombScatteringTest(unittest.TestCase):
    """Для кулоновского потенциала известны точные решения (резерфордовское
    ↪  рассеяние)."""

    def setUp(self):
        self.er = 100.0
        self.potential = CoulombPotential(1, 29)
        self.b = self.potential.z1 * self.potential.z2 * E2 / self.er
        self.g = RadialFunction(self.potential, self.er)
        self.solver = ClosestApproachSolver(self.g)
        self.rho = np.linspace(0.01, 2.0, 50) * self.b

    def test_closest_approach(self):
        expected = self.b / 2 + np.sqrt(self.b**2 / 4 + self.rho**2)
        np.testing.assert_allclose(self.solver.solve_many(self.rho), expected,
        ↪  rtol=1e-10)

    def test_gauss_mehler_angle(self):
```

```
        angle = GaussMehlerScatteringAngle(self.g, n_nodes=100)
        chi = angle.compute_many(self.rho, self.solver.solve_many(self.rho))
        expected = 2 * np.arctan(self.b / (2 * self.rho))
        np.testing.assert_allclose(chi, expected, atol=1e-4)

    def test_rutherford_cross_section(self):
        chi = 2 * np.arctan(self.b / (2 * self.rho))
        omega = DifferentialCrossSection(self.rho, chi)
        expected = (self.b / 4) ** 2 / np.sin(omega.theta / 2) ** 4
        np.testing.assert_allclose(omega.omega[1:-1], expected[1:-1], rtol=1e-2)


class KinematicsTest(unittest.TestCase):
    def setUp(self):
        self.collision = BinaryCollision(
            ParticleCatalog.get("p"), ParticleCatalog.get("Cu"), 0.1
        )

    def test_kinematic_factor_limits(self):
        m1, m2 = self.collision.m1, self.collision.m2
        self.assertAlmostEqual(float(self.collision.kinematic_factor(0.0)), 1.0)
        self.assertAlmostEqual(
            float(self.collision.kinematic_factor(np.pi)), ((m2 - m1) / (m1 + m2)) **
            ↪  2
        )

    def test_energy_conservation(self):
        chi = np.linspace(0.01, np.pi - 0.01, 50)
        theta1, theta2 = self.collision.lab_angles(chi)
        m1, m2 = self.collision.m1, self.collision.m2
        k1 = self.collision.kinematic_factor(theta1)
        k2 = 4 * m1 * m2 / (m1 + m2) ** 2 * np.cos(theta2) ** 2
        np.testing.assert_allclose(k1 + k2, 1.0, rtol=1e-12)


class ScreeningTest(unittest.TestCase):
    def test_coefficients_normalized(self):
        for function in (BOHR, MOLIERE, KR_C, ZBL):
            self.assertAlmostEqual(float(function(0.0)), 1.0, places=5)

    def test_screening_lengths(self):
        z1, z2 = 1, 29
        firsov = 0.8853 * A_BOHR * (z1**0.5 + z2**0.5) ** (-2 / 3)
        lindhard = 0.8853 * A_BOHR * (z1 ** (2 / 3) + z2 ** (2 / 3)) ** (-1 / 2)
        np.testing.assert_allclose(FirsovScreeningLength()(z1, z2), firsov, rtol=1e-4)
        np.testing.assert_allclose(LindhardScharffScreeningLength()(z1, z2), lindhard,
        ↪  rtol=1e-4)


class Variant4Test(unittest.TestCase):
    def test_variant_parameters(self):
        variant = Variant.from_number(4)
        potential = variant.build_potential()
        collision = variant.build_collision()
        self.assertIs(potential.screening_function, KR_C)
        self.assertIsInstance(potential.screening_length, FirsovScreeningLength)
        self.assertEqual((collision.projectile.symbol, collision.target.symbol), ("p",
        ↪  "Cu"))
        self.assertEqual(collision.energy, 0.1)

```

```
    def test_experiment(self):
        results = TwoBodyScatteringExperiment(Variant.from_number(4)).run()
        self.assertEqual(len(results.rho), 200)
        self.assertLess(abs(1 - results.closest_approach[-1] / results.rho_max), 1e-3)
        self.assertTrue(np.all(np.diff(results.chi) < 0))
        self.assertTrue(np.all((results.chi > 0) & (results.chi < np.pi)))
        self.assertTrue(np.all(results.cross_section1.omega > 0))


if __name__ == "__main__":
    unittest.main()
```

A.13 Модуль report/make_figures.py

```
"""Подготовка рисунков и таблиц для LaTeX-отчёта по практическому заданию № 1.

Запуск из каталога practical_1:  python report/make_figures.py
"""

import sys
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
import numpy as np  # noqa: E402

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from two_body import TwoBodyScatteringExperiment, Variant  # noqa: E402
from two_body.plotting import ResultsPlotter  # noqa: E402
from two_body.potentials import E2, CoulombPotential  # noqa: E402
from two_body.scattering import (  # noqa: E402
    ClosestApproachSolver,
    GaussMehlerScatteringAngle,
    RadialFunction,
)

REPORT_DIR = Path(__file__).resolve().parent
FIGURES_DIR = REPORT_DIR / "figures"
GENERATED_DIR = REPORT_DIR / "generated"
NODE_COUNTS = (10, 25, 50, 100, 200, 400)


def num(value: float, digits: int = 4) -> str:
    """Число с десятичной запятой для вставки в LaTeX."""
    return f"{value:.{digits}f}".replace(".", "{,}")


def sci(value: float) -> str:
    """Число в экспоненциальной записи: 3,21 \\cdot 10^{-5}."""
    mantissa, exponent = f"{value:.2e}".split("e")
    return f"${mantissa.replace('.', '{,}')} \\cdot 10^{{{{int(exponent)}}}}$"


class CoulombBenchmark:
```

```
    """Тестовый расчёт: кулоновский потенциал, для которого известно точное
    ↪  решение."""

    def __init__(self, z1: int = 1, z2: int = 29, relative_energy: float = 100.0):
        self.potential = CoulombPotential(z1, z2)
        self.b = z1 * z2 * E2 / relative_energy
        self.g = RadialFunction(self.potential, relative_energy)
        self.solver = ClosestApproachSolver(self.g)
        self.rho = np.linspace(0.01, 2.0, 200) * self.b
        self.closest_approach = self.solver.solve_many(self.rho)

    def exact_chi(self):
        return 2 * np.arctan(self.b / (2 * self.rho))

    def numerical_chi(self, n_nodes: int):
        angle = GaussMehlerScatteringAngle(self.g, n_nodes)
        return angle.compute_many(self.rho, self.closest_approach)

    def max_error(self, n_nodes: int) -> float:
        return float(np.max(np.abs(self.numerical_chi(n_nodes) - self.exact_chi())))


class ReportArtifacts:
    """Формирует рисунки и фрагменты таблиц отчёта."""

    def __init__(self, variant_number: int = 4):
        self.variant = Variant.from_number(variant_number)
        self.results = TwoBodyScatteringExperiment(self.variant).run()
        self.coulomb = CoulombBenchmark()

    def variant_figures(self) -> None:
        ResultsPlotter(self.results, FIGURES_DIR, dpi=200,
        ↪  show_titles=False).plot_all()

    def coulomb_figure(self) -> None:
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.8))
        rho = self.coulomb.rho / self.coulomb.b
        ax1.plot(rho, np.degrees(self.coulomb.exact_chi()), label="точное решение")
        ax1.plot(rho[::8], np.degrees(self.coulomb.numerical_chi(100))[::8], "o",
                 ms=4, label="Гаусс–Мелер, N = 100")
        ax1.set_xlabel("ρ / b")
        ax1.set_ylabel("χ, град")
        ax1.legend()
        errors = [self.coulomb.max_error(n) for n in NODE_COUNTS]
        ax2.loglog(NODE_COUNTS, errors, "o-", label="максимальная погрешность")
        ax2.loglog(NODE_COUNTS, errors[0] * (NODE_COUNTS[0] / np.array(NODE_COUNTS)) **
        ↪  2,
                   "--", color="gray", label="∝ 1/N²")
        ax2.set_xlabel("N")
        ax2.set_ylabel("max |Δχ|, рад")
        ax2.legend()
        for ax in (ax1, ax2):
            ax.grid(True, which="both", alpha=0.3)
        fig.tight_layout()
        fig.savefig(FIGURES_DIR / "test_coulomb.png", dpi=200)
        plt.close(fig)

    def convergence_table(self) -> None:
        reference = TwoBodyScatteringExperiment(self.variant, n_nodes=1600).run().chi
        rows = []
```

```
        for n in NODE_COUNTS:
            chi = TwoBodyScatteringExperiment(self.variant, n_nodes=n).run().chi
            rows.append(f"{n} & {sci(self.coulomb.max_error(n))} & "
                        f"{sci(float(np.max(np.abs(chi - reference))))} \\\\ \\hline")
        (GENERATED_DIR / "convergence_rows.tex").write_text("\n".join(rows) + "\n",
                                                            encoding="utf-8")

    def results_table(self) -> None:
        res = self.results
        rows = []
        for i in [0] + list(range(19, len(res.rho), 20)):
            rows.append(
                f"{num(res.rho[i], 5)} & {num(res.closest_approach[i], 5)} & "
                f"{num(np.degrees(res.chi[i]), 2)} & {num(np.degrees(res.theta1[i]),
                ↪  2)} & "
                f"{num(np.degrees(res.theta2[i]), 2)} \\\\ \\hline"
            )
        (GENERATED_DIR / "results_rows.tex").write_text("\n".join(rows) + "\n",
                                                        encoding="utf-8")

    def build(self) -> None:
        # Крупный шрифт: рисунки уменьшаются до ширины полосы набора
        plt.rcParams.update({"font.size": 16})
        FIGURES_DIR.mkdir(exist_ok=True)
        GENERATED_DIR.mkdir(exist_ok=True)
        self.variant_figures()
        self.coulomb_figure()
        self.convergence_table()
        self.results_table()


if __name__ == "__main__":
    ReportArtifacts().build()
    print(f"Рисунки: {FIGURES_DIR}\nТаблицы: {GENERATED_DIR}")
```